



TRAKYA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4502 YAPAY SİNİR AĞLARINA GİRİŞ
PROJE RAPORU

PROJE ADI:

Akıllı Park

PROJE EKİBİ:

1221602028 – İrem Su ERDEMİR

1221602043- Hatice Pınar YILMAZ

1221602098- Beyza YILMAZ

1221602641 – Yusuf GÜDE

1221602804 – Ulaş Ekrem EMİLİ

PROJE DANIŞMANI:

Dr. Öğr. Üyesi Turgut DOĞAN

Edirne -2026

İÇİNDEKİLER

1. GİRİŞ & PROJENİN TANITIMI.....	1
1.1. Akıllı Şehirlerde Otopark Problemi	1
1.2. Projenin Amacı ve Kapsamı	1
1.3. Parking Birmingham Veri Setinin Seçilme Gerekçesi.....	2
1.4. Projenin Özgün Yönleri.....	3
1.5. Hibrit Derin Öğrenme ve Pekiştirmeli Öğrenme Mimarisi.....	3
1.6. Gerçek Hayattaki Kullanım Senaryoları	4
1.7. Trafik Yoğunluğu ve Park Arama Davranışı.....	4
1.8. LSTM ve PPO Entegrasyonunun Teknik Gerekçesi.....	6
2. GEREKSİNİM ANALİZİ	7
2.1. Platform ve İşletim Sistemi Gereksinimleri	7
2.2. Kaynak Kısıtları ve Çözüm Önerileri	7
2.3. Çözülen Problemler ve Avantajlar	8
2.4. Hedef Kullanıcı Kitlesi.....	8
2.5. Kullanılan Teknolojiler ve Gerekçeleri.....	8
2.6. Destek Alınan Ancak Projede Yer Almayan Teknolojiler	9
3. TASARIM.....	10
3.1. Genel Sistem Mimarisi.....	10
3.2. Veri Ön İşleme ve Özellik Mühendisliği	10
3.3. LSTM Pipeline.....	11
3.4. RL Pipeline ve PPO Eğitim Mantığı.....	12
3.5. Durum Uzayı (State Space)	12
3.6. Eylem Uzayı (Action Space)	12
3.7. Ödül Fonksiyonu ve Reward Shaping.....	12

3.8.	Grid Environment ve Dynamic Occupancy	13
3.9.	Salınım Problemi ve Stabilizasyon	13
3.10.	Explainable AI ve Streamlit Dashboard	13
3.11.	Veri Akış Diyagramı ve Sequence Generation.....	14
3.12.	Static Goal vs Dynamic Goal Tartışması	15
3.13.	Action Masking ve Geçersiz Eylem Yönetimi	15
3.14.	DQN Eğitim Yapılandırması	15
3.15.	SWOT Analizi	16
4.	GELİŞTİRME	17
	Kurulum ve Çalıştırma Yönergeleri:	27
5.	TEST ve DOĞRULAMA	28
5.1.	LSTM Tahmin Performansı.....	28
5.2.	RL Ajan Performansı	29
5.3.	PPO vs DQN Analizi	31
5.4.	Baseline Karşılaştırması	31
5.5.	Oscillation ve Reward Shaping Etkisi.....	32
5.6.	Ajan Animasyonları ve Görsel Doğrulama	32
5.7.	Deney Tekrarlanabilirliği	33
6.	SONUÇ	34
6.1.	Gelecek Çalışmalar (Future Work).....	34
	KAYNAKLAR	35

1. GİRİŞ & PROJENİN TANITIMI

1.1. Akıllı Şehirlerde Otopark Problemi

Hızla büyüyen kentlerde araç sahipliğinin artması, sürücülerin park yeri ararken harcadıkları süreyi ve dolayısıyla trafik yoğunluğunu ciddi biçimde yükseltmektedir. Literatürde yapılan çalışmalar, şehir merkezlerinde dolaşan trafiğin önemli bir bölümünün boş park alanı aramaktan kaynaklandığını göstermektedir. Bu durum yalnızca zaman kaybına değil, yakıt tüketimi, karbon salınımı ve sürücü stresi gibi ek maliyetlere de yol açmaktadır. Akıllı şehir kavramı, sensör ağları, veri analitiği ve yapay zekâ bileşenlerini bir araya getirerek kentsel hizmetlerin verimliliğini artırmayı hedefler. Otopark yönetimi, akıllı şehir uygulamalarının en somut alanlarından biridir; çünkü doluluk bilgisinin doğru zamanında sunulması hem bireysel sürücü deneyimini hem de altyapı kullanım verimliliğini doğrudan etkiler.

Günümüzde birçok otopark işletmesi yalnızca anlık doluluk göstergesi sunmaktadır. Bu yaklaşım, sürücünün bir sonraki saat diliminde hangi bölgenin daha az yoğun olacağını öngörmesine yardımcı olmaz. Ayrıca mevcut navigasyon uygulamaları genellikle statik hedef noktalar üzerinden yönlendirme yapmakta; doluluk dinamiklerini ve geleceğe yönelik eğilimleri dikkate almamaktadır. Bu proje, bu boşluğu kapatmak amacıyla gerçek şehir otopark verisi üzerinde çalışan hibrit bir yapay zekâ mimarisi geliştirmiştir.

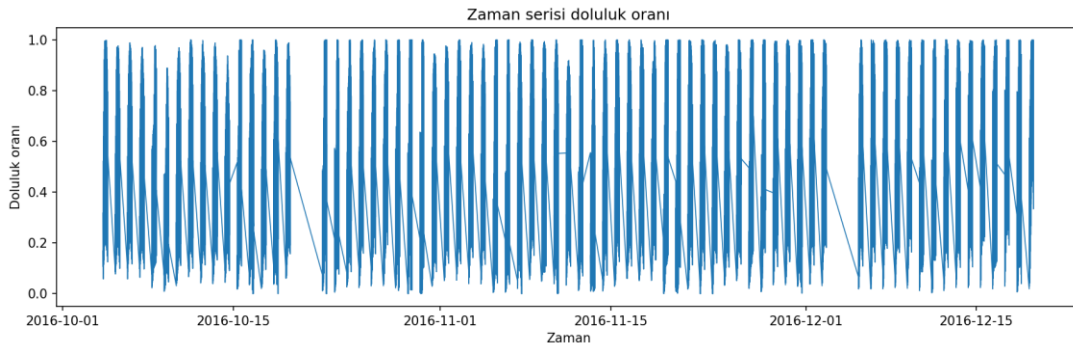
1.2. Projenin Amacı ve Kapsamı

Akıllı Park projesi, Trakya Üniversitesi Bilgisayar Mühendisliği Bölümü BLM4502 Yapay Sinir Ağlarına Giriş dersi kapsamında geliştirilmiştir. Projenin temel amacı, Parking Birmingham veri setinden elde edilen gerçek otopark doluluk kayıtlarını kullanarak kısa vadeli doluluk tahmini yapmak ve bu tahminleri pekiştirmeli öğrenme tabanlı bir yönlendirme ajanının durum uzayına entegre etmektir. Sistem, sürücüyü o anki ve tahmin edilen doluluk koşullarına göre en uygun (en düşük yoğunluklu) hedef otoparka yönlendirmeyi hedefler.

Proje kapsamında veri hazırlama, keşifsel veri analizi, LSTM tabanlı zaman serisi tahmini, tabular baseline modelleri (Random Forest, XGBoost), 15×15 ızgara tabanlı Gymnasium ortamında PPO ve DQN eğitimi, baseline karşılaştırmaları, FastAPI arka uç, React ön uç ve Streamlit karar destek paneli geliştirilmiştir. Tüm deney çıktıları `output/` ve `evaluation/reports/` klasörlerinde saklanmış; bu raporda yalnızca gerçek metrikler ve üretilmiş görseller kullanılmıştır.

1.3. Parking Birmingham Veri Setinin Seçilme Gerekçesi

Proje başlangıcında İzmir merkezli canlı veri toplama altyapısı değerlendirilmiş; ancak örneklem büyüklüğünün sınırlı olması, kesintisiz veri akışı için ek donanım gereksinimi ve öğrenci bütçesi kısıtları nedeniyle bu yaklaşımdan vazgeçilmiştir. Metodolojik tutarlılık, yeterli örneklem büyüklüğü ve deneylerin yeniden üretilebilirliği açısından Parking Birmingham veri seti tercih edilmiştir. Veri seti; SystemCodeNumber (otopark kimliği), Capacity, Occupancy ve LastUpdated alanlarını içermekte; çoklu otopark lokasyonları üzerinden saatlik/periodyik doluluk kayıtları sunmaktadır. Bu yapı, LSTM tabanlı zaman serisi tahmini ve RL ortamında dinamik doluluk haritası oluşturma için uygundur.



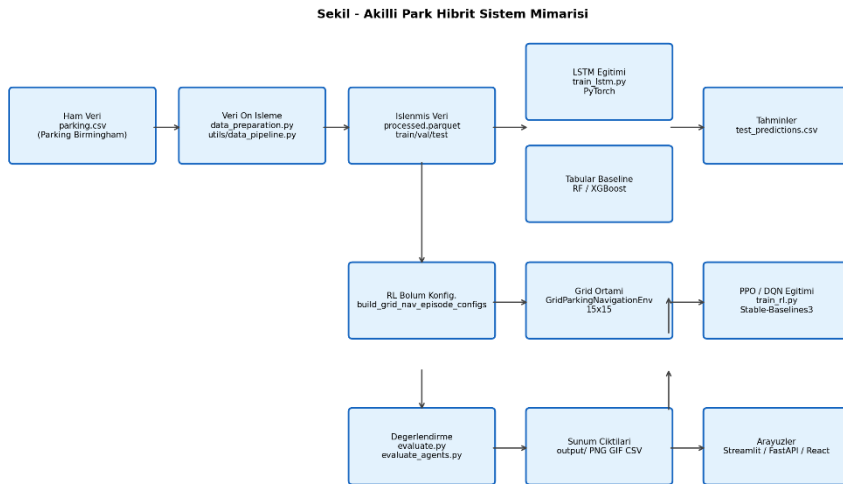
ŞEKİL 1.3. Parking Birmingham Veri Setinde Kronolojik Doluluk Oranı Zaman Serisi Grafiği

1.4. Projenin Özgün Yönleri

Literatürdeki birçok çalışma yalnızca doluluk sınıflandırması veya tahminine odaklanmaktadır. Bu projenin özgün değeri, tahmin modeli çıktıların PPO/DQN tabanlı RL ajanının gözlem vektörüne skaler kanal olarak bağlanmasıdır. Böylece sistem yalnızca mevcut doluluk haritasını değil, LSTM aggregate tahminini de dikkate alarak karar verir. Ayrıca maliyet duyarlı, bileşenleri ayrıştırılabilir (explainable) bir ödül fonksiyonu; Manhattan mesafe şekillendirme, salınım (oscillation) cezası, yüksek doluluk lotu geçiş cezası ve BFS optimaline yakınlık bonusu gibi mekanizmalar birlikte kullanılmıştır. Streamlit panelinde her RL adımının ödül bileşenleri kullanıcıya açıklanabilir biçimde sunulmaktadır.

1.5. Hibrit Derin Öğrenme ve Pekiştirmeli Öğrenme Mimarisi

Sistem mimarisi iki ana hattan oluşur. Birinci hat (denetimli öğrenme), `'utils/data_pipeline.py'` ile üretilen lag, rolling istatistik ve zaman özelliklerini kullanarak 12 zaman adımlık sliding window ile bir sonraki aggregate doluluk oranını tahmin eder. PyTorch tabanlı çok katmanlı LSTM mimarisi (`'utils/forecast_models.py'`) bu görevi üstlenir. İkinci hat (pekiştirmeli öğrenme), her benzersiz zaman damgası için bir RL bölümü oluşturur; lotlar hash tabanlı koordinatlarla 15×15 ızgara üzerine yerleştirilir ve hedef lot o anki en düşük doluluk oranına sahip alandır. LSTM tahmini `'lstm_aggregate_pred'` skaleri olarak gözleme eklenir; `'predicted_emptying_norm'` ise hedef lot doluluğunu temsil eder.



ŞEKİL 1.5. Akıllı Park Hibrit Sistem Mimarisi ve Veri Akış Diyagramı

PPO (Proximal Policy Optimization) algoritması, Stable-Baselines3 kütüphanesi ile 280.000 zaman adımı boyunca eğitilmiştir. VecNormalize ile gözlem ve ödül normalizasyonu uygulanmış; Monitor ile episode logları `logs/monitor/ppo/` altına yazılmıştır. DQN ile karşılaştırmalı deneyler aynı ortam ve hiperparametre yapılandırması altında yürütülmüştür.

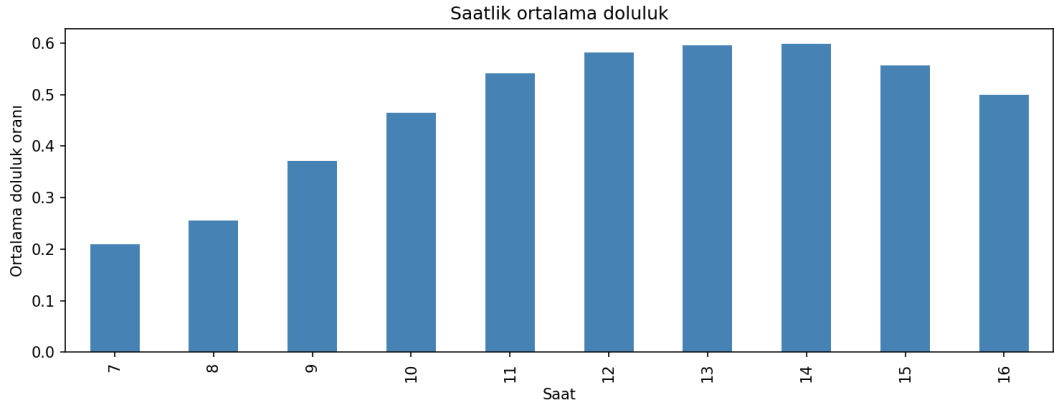
1.6. Gerçek Hayattaki Kullanım Senaryoları

- Alışveriş merkezi otoparklarında yoğun saatlerde sürücüyü alternatif giriş veya daha az dolu kata yönlendirme.
- Hastane kampüslerinde acil durum ziyaretçilerinin boş alana hızlı erişimi.
- Üniversite kampüslerinde ders saatlerine göre dinamik park önerisi.
- Belediye akıllı şehir platformlarına entegre edilebilecek API tabanlı karar destek servisi.
- Mobil uygulama veya navigasyon sistemlerine kısa vadeli doluluk tahmini beslemesi.

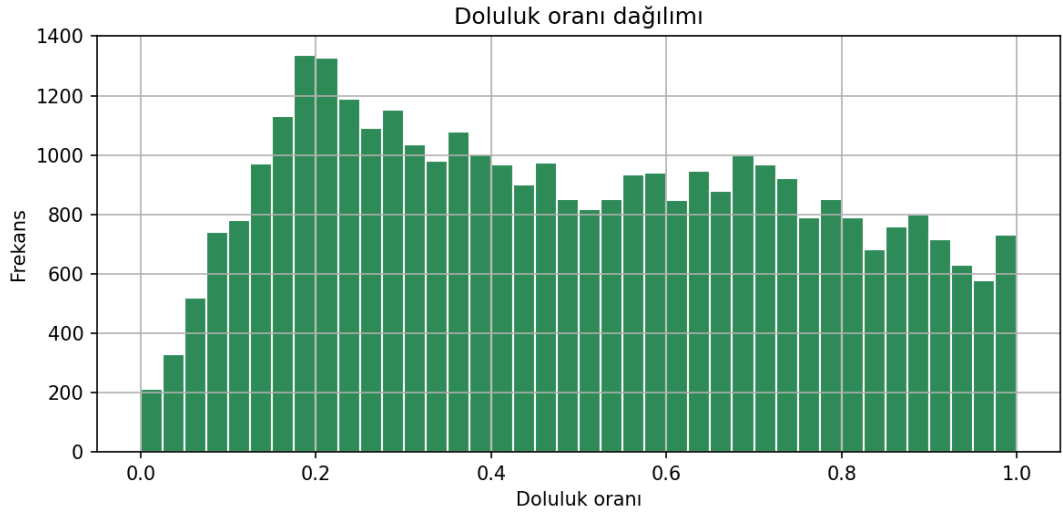
Geliştirilen FastAPI (`api/main.py`) ve React (`frontend/`) bileşenleri, Streamlit panelinin yanı sıra üretim ortamına yakın bir dağıtım senaryosunu destekleyecek şekilde tasarlanmıştır.

1.7. Trafik Yoğunluğu ve Park Arama Davranışı

Keşifsel veri analizi sonuçları, otopark doluluk oranlarının gün içinde belirgin periyodik desenler gösterdiğini ortaya koymuştur. Saatlik ortalama doluluk grafiği (`output/2\_saatlik\_ortalama.png`) sabah ve öğle saatlerinde pikler; gece saatlerinde düşüş eğilimi sergilemektedir. Hafta içi ve hafta sonu karşılaştırması ise kullanım alışkanlıklarının farklılaştığını göstermektedir. Bu bulgular, yalnızca anlık doluluk yerine zaman serisi tahmininin neden gerekli olduğunu destekler: sürücü hedefe varmadan önce doluluk değişebilir.



ŞEKİL 1.7.1. Saatlik Ortalama Otopark Doluluk Oranı Grafiği



ŞEKİL 1.7.2. Doluluk Oranı Dağılım Histogram Grafiği

1.8. LSTM ve PPO Entegrasyonunun Teknik Gerekçesi

LSTM modelinin aggregate seri üzerinde eğitilmesi, lot bazlı gürültüyü yumuşatarak genel şehir doluluk eğilimini yakalar. RL ajanına aktarılan `'lstm_aggregate_pred'` skaleri, ajanın 'gelecekte genel doluluk artacak mı?' sorusuna kısa vadeli cevap almasını sağlar. Hedef lot seçimi ise o anki gerçek doluluk oranları üzerinden en düşük yoğunluklu alan belirlenerek yapılır; böylece tahmin ve anlık ölçüm birlikte kullanılır. Bu entegrasyon, künyede planlanan hibrit mimarinin kod tabanında (`env/grid_navigation_env.py`, satır 105–117 `GridNavEpisodeConfig`) somutlaştırılmasıdır.

PPO algoritmasının DQN'e tercih edilmesi, yüksek boyutlu gözlem uzayında politika gradyan yöntemlerinin daha az instabilite göstermesi beklentisine dayanmaktadır. Deney sonuçları bu beklentiye doğrulamıştır: PPO %92,5 başarı oranına ulaşırken DQN %27,5 ile sınırlı kalmıştır. Entegrasyon sayesinde sistem yalnızca reaktif değil, proaktif karar verme kapasitesine kavuşmuştur.

2. GEREKSİNİM ANALİZİ

2.1. Platform ve İşletim Sistemi Gereksinimleri

Akıllı Park, masaüstü geliştirme ve sunum ortamında çalışacak şekilde tasarlanmış bir yazılım projesidir. Python tabanlı makine öğrenmesi bileşenleri Windows, Linux ve macOS işletim sistemlerinde sanal ortam içinde çalıştırılabilir. Web arayüzleri tarayıcı tabanlıdır; React geliştirme sunucusu ve Streamlit uygulaması yerel ağ veya localhost üzerinden erişilebilir. FastAPI arka uç servisi sunucu/ masaüstü platformlarında uvicorn ile barındırılabilir.

Gereksinim	Değer / Açıklama
Platform	Masaüstü (geliştirme), sunucu (API), tarayıcı (UI)
İşletim sistemi	Windows 10+, Ubuntu 20.04+, macOS 12+
Python	3.10+ (3.11 önerilir)
Node.js	18+ (React frontend için)
Bellek	Minimum 8 GB RAM (RL eğitimi için 16 GB önerilir)
Disk	~2 GB (veri + modeller + çıktılar)
GPU	Opsiyonel (PyTorch CPU ile çalışır)

TABLO 2.1. Platform ve Teknoloji Gereksinimleri

2.2. Kaynak Kısıtları ve Çözüm Önerileri

Proje geliştirme sürecinde karşılaşılan başlıca kısıtlar şunlardır: (1) Canlı İzmir otopark verisine erişim ve kenar cihaz donanımı bütçesi — Parking Birmingham statik veri seti ile çözüldü. (2) RL eğitim süresi — 280.000 timestep hedefi makineye göre saatler sürebilir; `--timesteps` parametresi ile hızlı deneme yapılabilir. (3) Yüksek boyutlu gözlem uzayı (1127 boyut) — VecNormalize ve ödül kırpma (`GRID\_REWARD\_CLIP=550`) ile stabilizasyon sağlandı. (4) DQN'in uzun episode timeout davranışı — PPO ile karşılaştırmalı analiz ve reward shaping ile iyileştirme stratejileri uygulandı.

2.3. Çözülen Problemler ve Avantajlar

Proje; körlemesine park arama süresini azaltma, trafik yoğunluğu azaltma potansiyeli ölçümü (random baseline'a göre %86,83 trafik azaltma proxy değeri), kısa vadeli doluluk tahmini ve açıklanabilir RL kararları sunma problemlerine çözüm getirir. Mevcut yalnızca-anlık-gösterge çözümlerine kıyasla geleceğe yönelik tahmin entegrasyonu, çoklu algoritma karşılaştırması ve interaktif demo arayüzleri projeyi farklılaştırır.

2.4. Hedef Kullanıcı Kitlesi

Birincil kullanıcılar: şehir içi sürücüler, otopark işletmecileri, akıllı şehir yönetim birimleri. İkincil kullanıcılar: veri bilimciler, BLM4502 öğrenci/jüri değerlendiricileri, demo izleyicileri. Streamlit paneli tek kullanıcı prototip olarak tasarlanmıştır; FastAPI + React mimarisi çoklu eşzamanlı istek için ölçeklenebilir altyapı sunar.

2.5. Kullanılan Teknolojiler ve Gerekçeleri

Teknoloji	Avantaj	Dezavantaj
Python	Veri bilimi ekosistemi, okunabilirlik, ekip işbirliği	Yorumlama hızı C++'a göre düşük
PyTorch	LSTM/GRU/Transformer eğitimi, dinamik graf	TensorFlow'a göre farklı API
Stable-Baselines3	PPO/DQN hazır implementasyon, VecNormalize	Özel ortam entegrasyonu gerekir
Gymnasium	Standart RL ortam API'si	Simülasyon gerçek trafik değildir
Pandas/NumPy	Veri manipülasyonu, vektör işlemleri	Büyük veride bellek baskısı

Scikit-learn	Scaler, RF baseline, metrikler	Derin öğrenme değildir
Matplotlib	EDA ve RL grafikleri	İnteraktif görselleştirme sınırlı
Streamlit	Hızlı ML dashboard prototipi	Kurumsal UI esnekliği düşük
FastAPI	REST API, otomatik OpenAPI	Ek deployment yapılandırması
React (Vite)	Modern SPA ön uç	Node.js bağımlılığı

TABLO 2.5. Kullanılan Teknolojilerin Avantaj ve Dezavantaj Analizi

Python, projenin tüm bileşenlerinde ortak dil olarak seçilmiştir; veri hazırlama (`data\_preparation.py`), model eğitimi (`train\_lstm.py`, `train\_rl.py`) ve değerlendirme (`evaluate.py`) scriptleri aynı sanal ortamda çalışır. TensorFlow/Keras yerine PyTorch tercih edilmiştir; proje kodunda `torch.nn` modülleri doğrudan kullanılmaktadır. requirements.txt dosyası tüm bağımlılıkları sürüm aralıkları ile tanımlar.

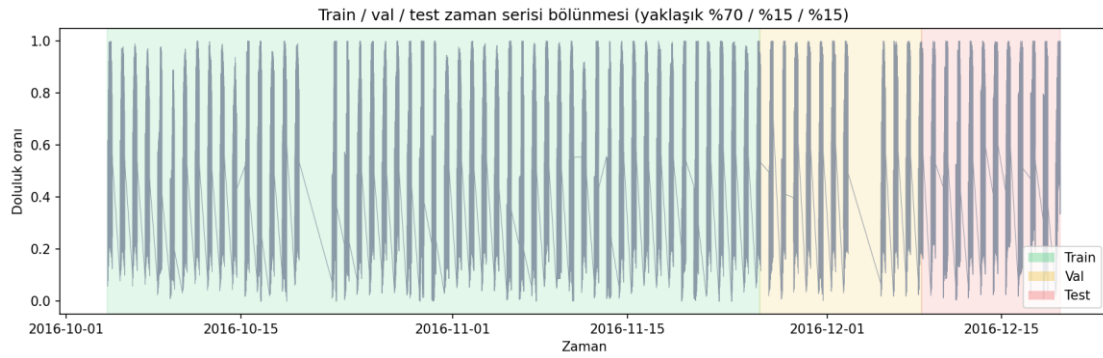
2.6. Destek Alınan Ancak Projede Yer Almayan Teknolojiler

- **Raspberry Pi / IoT kenar cihaz** — bütçe nedeniyle canlı veri toplama yerine Birmingham verisi.
- **Gerçek zamanlı trafik API** — veri setinde trafik akışı bulunmamaktadır.
- **ROS / SUMO simülatörü** — grid tabanlı özel ortam tercih edilmiştir.

3. TASARIM

3.1. Genel Sistem Mimarisi

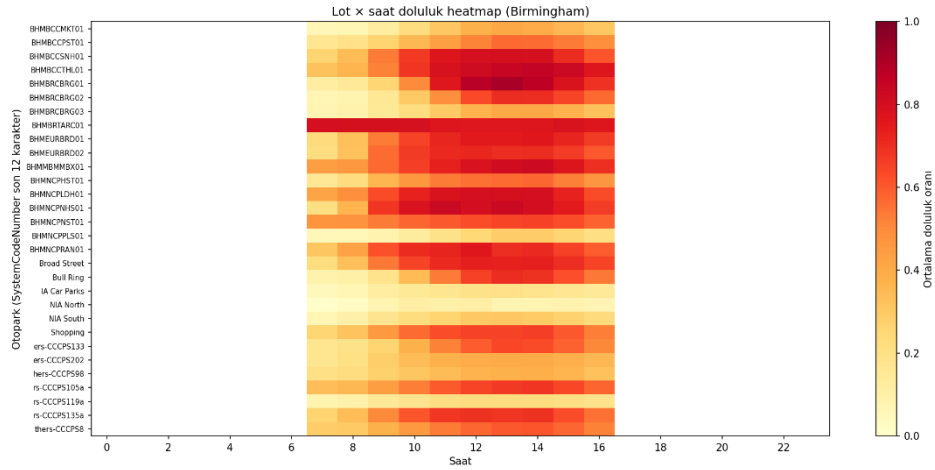
Sistem beş katmandan oluşur: (1) Veri katmanı — ham CSV, işlenmiş parquet ve bölünmüş CSV dosyaları. (2) Tahmin katmanı — LSTM ve tabular modeller. (3) RL simülasyon katmanı — GridParkingNavigationEnv. (4) Değerlendirme katmanı — metrik toplama, grafik ve GIF üretimi. (5) Sunum katmanı — Streamlit, FastAPI, React. Veri akışı tek yönlüdür: ham veri → özellik mühendisliği → tahmin → bölüm konfigürasyonu → ajan eğitimi → raporlama.



ŞEKİL 3.1. Zaman Bazlı Train/Validation/Test Bölme Stratejisi (%70/%15/%15)

3.2. Veri Ön İşleme ve Özellik Mühendisliği

Veri hattı `data\_preparation.py` ve `utils/data\_pipeline.py` modülleri ile yürütülür. Yinelenen kayıtlar silinir; Occupancy [0, Capacity] aralığı doğrulanır; LastUpdated datetime'a dönüştürülür. Lot bazında IQR aykırı değer filtresi (çarpan=3.0, minimum 30 örnek/lot) uygulanır. Lag özellikleri (1, 3, 6, 12, 24 adım), rolling mean/variance (7 ve 24 pencere), saat, gün, hafta içi/sonu ve mevsimsel sin/cos özellikleri üretilir. MinMaxScaler yalnızca eğitim kümesinde fit edilir.



ŞEKİL 3.2. Otopark Lotları × Saat Dilimi Doluluk Oranı Haritası

3.3. LSTM Pipeline

Aggregate seri: tüm lotların aynı zaman damgasındaki ortalama doluluk oranı. Sliding window: LSTM_TIME_STEP=12 ile 12 ardışık zaman adımı giriş, bir sonraki adım hedef. Mimari: 2 katmanlı LSTM (hidden=64), dropout=0.2, linear head. Eğitim: Adam optimizer, ReduceLROnPlateau, early stopping (patience=8), isteğe bağlı weighted L1 loss. Optuna ile hiperparametre araması desteklenir.

Parametre	Değer
Zaman penceresi (time_step)	12
Epoch (varsayılan)	80
Batch size	32
Gizli birim	64
Katman sayısı	2
Dropout	0.2
Optimizer	Adam
Kayıp fonksiyonu	L1 (weighted opsiyonel)

TABLO 3.3. LSTM Model Hiperparametreleri

3.4. RL Pipeline ve PPO Eğitim Mantığı

Her benzersiz LastUpdated zaman damgası bir RL bölümü oluşturur. 'build_grid_nav_episode_configs' fonksiyonu lot koordinatlarını, doluluk oranlarını, hedef hücreyi (en düşük doluluklu lot) ve LSTM skalerlerini paketler. PPO eğitimi 'train_rl.py' içinde MlpPolicy, learning_rate=2.5e-4, n_steps=2048, batch_size=128, gamma=0.99, clip_range=0.2 ile yürütülür. 280.000 toplam timestep hedeflenir; Monitor CSV'ler 'logs/monitor/' altında saklanır.

3.5. Durum Uzayı (State Space)

Gözlem boyutu: $5 \times 15 \times 15 + 2 = 1127$ (float32, [0,1]). Kanallar: (1) duvar maskesi, (2) park alanı maskesi, (3) hedef hücresi, (4) ajan konumu, (5) doluluk ısı haritası. Skalerler: lstm_aggregate_pred (LSTM tahmini), predicted_emptying_norm (hedef lot doluluğu vekili). Bu yapı künyede tanımlanan hibrit RL durum kodlamasını gerçekleştirir.

3.6. Eylem Uzayı (Action Space)

Discrete(4): 0=yukarı, 1=aşağı, 2=sol, 3=sağ. Duvar veya ızgara sınırına çarpışmada ajan yerinde kalır ve ek ceza alır. Geçersiz eylem indeksi yalnızca adım maliyeti uygular. Geleneksel action masking kullanılmamıştır; bunun yerine çarpışma cezası ve Manhattan shaping ile yönlendirme sağlanır.

3.7. Ödül Fonksiyonu ve Reward Shaping

Bileşen	Sabit	Açıklama
Zaman cezası	GRID_STEP_COST = 1.25	Her adımda -1.25
Manhattan shaping	SCALE = 4.5	Hedefe yaklaşınca pozitif
İlk ziyaret bonusu	+0.08	Yeni hücre keşfi
Tekrar ziyaret cezası	-4.0	Gereksiz dolaşım
Salınım cezası	GRID_LOOP_PENALTY = -6.0	8 adımlık A↔B döngü
Hedef bonusu	+300.0	Terminal başarı

Timeout cezası	-200.0	200 adım sınırı
Yol verimliliği	SCALE = 45.0	BFS optimaline yakınlık
Yüksek doluluk	threshold=0.88, ceza=22	Dolu lot geçişi
Ödül kırpma	GRID_REWARD_CLIP = 550	Stabilizasyon

TABLO 3.7. Ödül Fonksiyonu Bileşenleri (ml_config.py / reward_utils.py)

3.8. Grid Environment ve Dynamic Occupancy

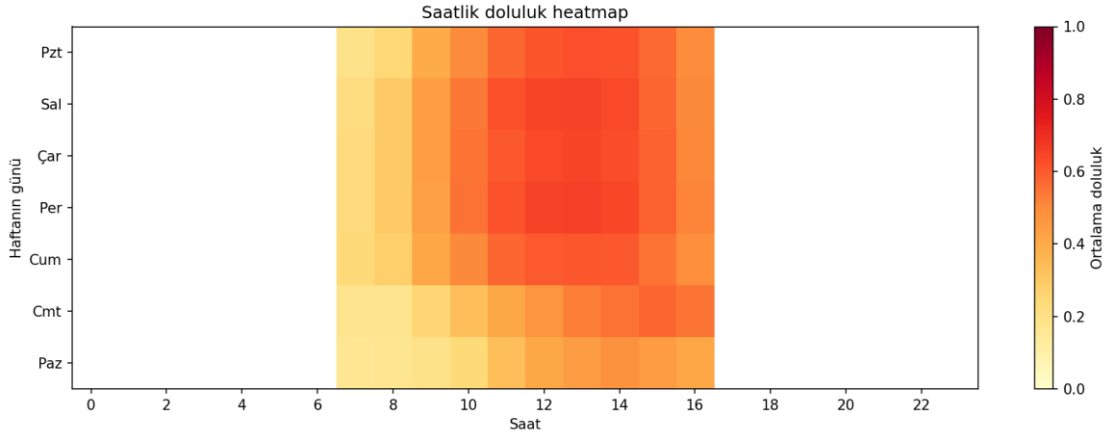
GridParkingNavigationEnv 15×15 boyutundadır; duvar hücreleri sabittir, park lotları doluluk oranına göre renklendirilir. Senaryo parametresi (`scenario`): low (×0.55), medium (×1.0), high (×1.35), dynamic (rastgele güncelleme). Eğitimde statik hedef lot (bölüm başında belirlenen en düşük doluluklu lot) tercih edilmiştir. Dynamic goal senaryoları kararlılığı düşürebilir; bu nedenle ana deneyler sabit hedef yapılandırması ile yürütülmüştür.

3.9. Sahnım Problemi ve Stabilizasyon

Grid ortamlarında ajanların A↔B zigzag hareketi (oscillation) yaygın bir sorundur. Projede GRID_LOOP_WINDOW=8 penceresinde tekrarlayan konum deseni tespit edilerek GRID_LOOP_PENALTY=-6.0 uygulanır. Tekrar ziyaret cezası (-4.0) ve düşük ilk ziyaret bonusu (0.08) birlikte gereksiz keşfi sınırlar. PPO eğitiminde ent_coef=0.0 ayarlanarak rastgele keşif azaltılmış; VecNormalize ile ödül ölçeklemesi stabilize edilmiştir. Monitor loglarında PPO'nun son bölümlerde +300 ile +360 arası ödül ve 4–18 adım uzunluğu gözlemlenmiştir.

3.10. Explainable AI ve Streamlit Dashboard

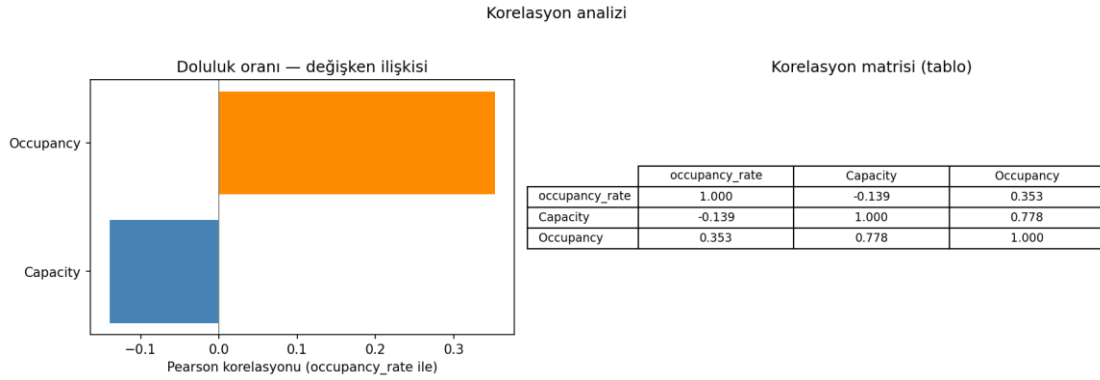
Streamlit uygulaması (`ui\_streamlit/app.py`) beş sekmeden oluşur: (1) Zaman serisi tahmin ve MAE/RMSE/MAPE/R² metrikleri, (2) EDA grafikleri, (3) Sistem özeti dashboard, (4) RL simülasyon ve adım adım grid görselleştirme, (5) Karar destek ve açıklanabilir öneri. `\_explain\_reward` fonksiyonu her adımın ödülünü Türkçe açıklamaya dönüştürür. GIF çıktıları (`ppo\_agent\_explained.gif`) canlı metrik paneli ile birlikte sunulur.



ŞEKİL 3.10. Gün × Saat Doluluk Isı Haritası (RL senaryo analizi için)

3.11. Veri Akış Diyagramı ve Sequence Generation

LSTM sequence generation `utils/lstm\_core.py` modülündeki `make\_sequences` fonksiyonu ile gerçekleştirilir. Ardışık 12 zaman adımı özellik matrisi giriş; bir sonraki aggregate occupancy_rate hedef olarak atanır. RL tarafında her timestamp için `build\_grid\_nav\_episode\_configs` çağrılır; lot koordinatları `stable\_parking\_coordinates` ile hash tabanlı yerleştirilir. Tahmin CSV varsa LSTM skaleri bölüme enjekte edilir.



ŞEKİL 3.11. Özellikler Arası Korelasyon Matrisi

3.12. Static Goal vs Dynamic Goal Tartışması

Dynamic goal senaryosunda hedef lot bölüm ortasında değişebilir; bu durum Manhattan shaping sinyalinin zayıflatır ve ajanın osilasyon yapma eğilimini artırır. Static training yaklaşımında hedef bölüm başında belirlenir ve sabit kalır; PPO eğitim loglarında bu yapılandırmanın daha monoton artan reward trendi ürettiği gözlemlenmiştir. Dynamic senaryo ('SCENARIO_DYNAMIC') stres testi olarak 'scenario=' parametresi ile desteklenir; ana metrikler static hedef konfigürasyonu üzerinden raporlanmıştır.

3.13. Action Masking ve Geçersiz Eylem Yönetimi

Projede klasik RL action masking (geçersiz eylemlerin policy softmax'ından çıkarılması) uygulanmamıştır. Bunun yerine duvar çarpışmasında ajan konumu değişmez ve '- GRID_STEP_COST - wall_penalty' cezası uygulanır. Park alanı maskesi gözlemin ikinci kanalında yer alır; ajan dolu lot hücresinden geçtiğinde GRID_HIGH_OCC_STEP_PENALTY devreye girer. Bu yaklaşım, maskesiz ortamlarda öğrenmeyi zorlaştırırsa da gerçek dünya navigasyonunda 'duvara çarpma' metaforunu korumaktadır.

3.14. DQN Eğitim Yapılandırması

Parametre	Değer
learning_rate	1e-4
buffer_size	150.000
learning_starts	2000
batch_size	64
gamma	0.99
exploration_fraction	0.25
exploration_final_eps	0.05

TABLO 3.14. DQN Hiperparametreleri (train_rl.py)

3.15. SWOT Analizi

Kategori	Değerlendirme
Güçlü yönler	Hibrit DL+RL, gerçek şehir verisi, açıklanabilir ödül, çoklu UI
Zayıf yönler	Statik veri, gerçek zamanlı API yok, trafik verisi eksik
Fırsatlar	Belediye/AVM/hastane entegrasyonu, akıllı şehir projeleri
Tehditler	Veri güncelliği, sensör erişim kısıtları, genellenebilirlik

TABLO 3.15. Proje SWOT Analizi

4. GELİŞTİRME

```
47 def validate_occupancy(df: pd.DataFrame) -> pd.DataFrame:
48     """
49     Occupancy fiziksel olarak mantıklı olsun.
50     Capacity 0 veya negatif satırlar elenir; Occupancy [0, Capacity] aralığına çekilmez,
51     aralık dışındakiler silinir (veri hatası varsayımı).
52     """
53     before = len(df)
54     cap = pd.to_numeric(df["Capacity"], errors="coerce")
55     occ = pd.to_numeric(df["Occupancy"], errors="coerce")
56     df = df.copy()
57     df["Capacity"] = cap
58     df["Occupancy"] = occ
59
60     df = df.dropna(subset=["Capacity", "Occupancy"])
61     df = df[df["Capacity"] > 0]
62     df = df[(df["Occupancy"] >= 0) & (df["Occupancy"] <= df["Capacity"])]
63
64     print(f"[data_prep] Occupancy doğrulama: {before} -> {len(df)} satır")
65     return df
```

ŞEKİL 4.1. Otopark Doluluk Verisi Doğrulama ve Temizleme

Şekil 4.1.’deki fonksiyonda Capacity ve Occupancy sütunları sayısal formata çevrilip geçersiz değerler ayıklanmıştır. Ardından kapasitesi 0’dan küçük/eşit olan, doluluğu negatif olan veya doluluğu kapasiteyi aşan kayıtlar silinerek analiz için tutarlı bir veri seti elde edilmiştir.

```
79 def split_by_timestamps(
80     df: pd.DataFrame,
81     train_ratio: float = 0.70,
82     val_ratio: float = 0.15,
83 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
84     """
85     Zaman serisini bozmadan böler: önce benzersiz LastUpdated sıralar,
86     sonra her satır kendi zaman damgasına göre tek bir split’e düşer.
87
88     Not: Satır sayısına göre değil, benzersiz zaman sayısına göre oran uygulanır;
89     böylece aynı anda çekilmiş çoklu otopark okumaları hep aynı parçada kalır.
90     """
91     unique_times = df["LastUpdated"].drop_duplicates().sort_values(kind="mergesort")
92     n_t = len(unique_times)
93     if n_t < 3:
94         raise ValueError(f"En az 3 benzersiz zaman gerekli (train/val/test), gelen: {n_t}")
95
96     # Kesme indeksleri: önce oransal, sonra boş küme kalmaması için sıkıştır
97     idx_train = int(n_t * train_ratio)
98     idx_val_end = int(n_t * (train_ratio + val_ratio))
99     idx_train = max(1, min(idx_train, n_t - 2))
100     idx_val_end = max(idx_train + 1, min(idx_val_end, n_t - 1))
101
102     train_times = set[Any](unique_times.iloc[:idx_train])
103     val_times = set[Any](unique_times.iloc[idx_train:idx_val_end])
104     test_times = set[Any](unique_times.iloc[idx_val_end:])
105
106     if not test_times:
107         # Çok seyrek veride son zamanı teste al
108         last_t = unique_times.iloc[-1]
109         test_times = {last_t}
110         val_times.discard(last_t)
111         train_times.discard(last_t)
112
113     train_df = df[df["LastUpdated"].isin(train_times)].copy()
114     val_df = df[df["LastUpdated"].isin(val_times)].copy()
115     test_df = df[df["LastUpdated"].isin(test_times)].copy()
116
117     print(
118         f"[data_prep] Zaman bazlı split – benzersiz zaman: {n_t}, "
119         f"train/val/test zaman sayıları: {len(train_times)}/{len(val_times)}/{len(test_times)}"
120     )
```

ŞEKİL 4.2. Zaman Serisine Uygun Train/Validation/Test Bölme

Şekil 4.2.’de veri, zaman sırası bozulmadan LastUpdated alanına göre train/validation/test kümelerine ayrılmıştır. Benzersiz zaman damgaları üzerinden bölme yapılarak aynı ana ait kayıtların farklı kümelere dağılması engellenmiş, böylece veri sızıntısı riski azaltılıp model değerlendirmesi daha güvenilir hale getirilmiştir.

```
40 LAG_STEPS = (1, 3, 6, 12, 24)
41 ROLL_WINDOWS = (7, 24)
42
43 FEATURE_COLS_FOR_SCALING = [
44     "occupancy_rate",
45     "lag_1",
46     "lag_3",
47     "lag_6",
48     "lag_12",
49     "lag_24",
50     "roll_mean_7",
51     "roll_mean_24",
52     "roll_var_7",
53     "roll_var_24",
54     "hour",
55     "day_of_week",
56     "is_weekend",
57     "month_sin",
58     "month_cos",
59     "day_of_year_sin",
60     "day_of_year_cos",
61 ]
```

ŞEKİL 4.3. Özellik Mühendisliği ve Ölçeklenecek Değişkenlerin Tanımlanması

Şekil 4.3.’te modelin geçmiş ve dönemsel örüntüleri öğrenebilmesi için gecikmeli (lag), hareketli ortalama/varyans (roll) ve takvim tabanlı zaman özellikleri bir araya getirilmiştir. Ayrıca FEATURE_COLS_FOR_SCALING listesi ile hangi sayısal değişkenlerin standartlaştırılacağı açıkça belirlenerek eğitim öncesi tutarlı bir ön işleme adımı oluşturulmuştur.

```

84 def _split_masks_by_timestamp(
85     timestamps: pd.Series,
86     train_ratio: float = 0.70,
87     val_ratio: float = 0.15,
88 ) -> pd.Series:
89     """Her satıra train/val/test etiketi; kesimler benzersiz zaman ekseninde (veri sızıntısı yok)."""
90     unique_times = timestamps.drop_duplicates().sort_values(kind="mergesort")
91     n_t = len(unique_times)
92     if n_t < 3:
93         raise ValueError(f"En az 3 benzersiz zaman gerekli, gelen: {n_t}")
94
95     idx_train = int(n_t * train_ratio)
96     idx_val_end = int(n_t * (train_ratio + val_ratio))
97     idx_train = max(1, min(idx_train, n_t - 2))
98     idx_val_end = max(idx_train + 1, min(idx_val_end, n_t - 1))
99
100     train_times = set[Any](unique_times.iloc[:idx_train])
101     val_times = set[Any](unique_times.iloc[idx_train:idx_val_end])
102     test_times = set[Any](unique_times.iloc[idx_val_end:])
103
104     if not test_times:
105         last_t = unique_times.iloc[-1]
106         test_times = {last_t}
107         val_times.discard(last_t)
108         train_times.discard(last_t)
109
110     def _label(ts: pd.Timestamp) -> str:
111         if ts in train_times:
112             return "train"
113         if ts in val_times:
114             return "val"
115         if ts in test_times:
116             return "test"
117         return "train"

```

ŞEKİL 4.4. Zaman Damgasına Göre Sızıntısız Veri Etiketleme

Şekil 4.4.’teki fonksiyon, benzersiz zaman damgalarını kronolojik sırada train/validation/test aralıklarına bölerek her kayda uygun küme etiketini atar. Aynı zaman damgasına sahip örneklerin tek bir kümede kalması sağlandığı için veri sızıntısı önlenir ve modelin değerlendirme sonuçları daha güvenilir olur.

```

235 def build_processed_dataset(
236     raw_path: Path | None = None,
237     output_path: Path | None = None,
238     scaler_kind: ScalerKind | None = None,
239     outlier_filter: bool | None = None,
240 ) -> pd.DataFrame:
241     ensure_all_standard_dirs()
242     raw_path = raw_path or (DATA_RAW / "parking.csv")
243     output_path = output_path or (DATA_PROCESSED / "processed.parquet")
244
245     if not raw_path.exists():
246         raise FileNotFoundError(f"Ham veri yok: {raw_path}")
247
248     df = _load_raw(raw_path)
249     df = apply_outlier_filter(df, enabled=outlier_filter)
250     df["split"] = _split_masks_by_timestamp(df["LastUpdated"])
251     df = _add_per_lot_history_features(df)
252
253     sk = scaler_kind or str(FEATURE_SCALER).strip().lower()
254     if sk not in ("minmax", "standard"):
255         raise ValueError(f"Geçersiz scaler_kind: {sk!r} (minmax | standard)")
256     out, _ = _scale_train_only(df, MODELS_DIR, sk) # type: ignore[arg-type]
257
258     output_path.parent.mkdir(parents=True, exist_ok=True)
259     out.to_parquet(output_path, index=False)
260     print(f"[pipeline] Parquet yazıldı: {output_path} ({len(out)} satır)")
261     return out

```

ŞEKİL 4.5. Uçtan Uca İşlenmiş Veri Seti Oluşturma Pipeline’ı

Şekil 4.5.’teki fonksiyon ham veriyi okuyup aykırı değer filtreleme, zaman bazlı train/validation/test etiketleme, geçmişe dayalı özellik üretimi ve ölçekleme adımlarını tek akışta uygular. Son aşamada işlenmiş veri parquet formatında kaydedilerek modelleme için tekrar üretilebilir ve standart bir veri seti hazırlanmış olur.

```
15 class OccupancyLSTM(nn.Module):
16     """Çok katmanlı LSTM + dropout + doğrusal başlık."""
17
18     def __init__(
19         self,
20         input_dim: int,
21         hidden: int = 64,
22         num_layers: int = 2,
23         dropout: float = 0.2,
24     ):
25         super().__init__()
26         self.rnn_type = "lstm"
27         self.lstm = nn.LSTM(
28             input_dim,
29             hidden,
30             num_layers,
31             batch_first=True,
32             dropout=dropout if num_layers > 1 else 0.0,
33         )
34         self.dropout = nn.Dropout(dropout)
35         self.head = nn.Linear(hidden, 1)
36
37     def forward(self, x: torch.Tensor) -> torch.Tensor:
38         y, _ = self.lstm(x)
39         last = self.dropout(y[:, -1, :])
40         return self.head(last).squeeze(-1)
```

ŞEKİL 4.6. Çok Katmanlı LSTM ile Doluluk Tahmin Modeli

Şekil 4.6.’daki sınıfta, zaman serisi doluluk verisini öğrenmek için çok katmanlı bir LSTM mimarisi tanımlanmıştır. LSTM çıkışına dropout uygulanıp son zaman adımı doğrusal katmana verilerek tek değerlik doluluk tahmini üretilmiştir.

```

53 def _train_loop(
54     model: nn.Module,
55     device: torch.device,
56     train_loader: DataLoader,
57     X_val_t: torch.Tensor,
58     y_val_t: torch.Tensor,
59     epochs: int,
60     patience: int,
61     lr: float,
62     weighted_loss: bool,
63     log_prefix: str = "[LSTM]",
64 ) -> tuple[dict[str, torch.Tensor], float]:
65     opt = torch.optim.Adam(model.parameters(), lr=lr)
66     scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
67         opt, mode="min", factor=0.5, patience=3, min_lr=1e-6
68     )
69     loss_fn = nn.L1Loss()
70
71     best_val = float("inf")
72     stale = 0
73     best_state: dict[str, torch.Tensor] | None = None
74
75     for epoch in range(epochs):
76         model.train()
77         for xb, yb in train_loader:
78             xb = xb.to(device)
79             yb = yb.to(device)
80             opt.zero_grad()
81             pred = model(xb)
82             if weighted_loss:
83                 loss = _batch_weighted_l1(pred, yb, xb)
84             else:
85                 loss = loss_fn(pred, yb)
86             loss.backward()
87             opt.step()
88
89         model.eval()
90         with torch.no_grad():
91             vpred = model(X_val_t)
92             vloss = float(loss_fn(vpred, y_val_t).item())
93         scheduler.step(vloss)
94         print(f"{log_prefix} epoch {epoch + 1}/{epochs} val_mae_mm: {vloss:.6f}")
95
96         if vloss < best_val - 1e-6:
97             best_val = vloss
98             stale = 0
99             best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}
100         else:
101             stale += 1
102             if stale >= patience:
103                 print(f"{log_prefix} early stopping")
104                 break
105
106         if best_state:
107             model.load_state_dict(best_state)
108     assert best_state is not None
109     return best_state, best_val

```

ŞEKİL 4.7. LSTM Eğitim Döngüsü, Doğrulama Takibi ve Erken Durdurma

Şekil 4.7.'de model, her epoch'ta eğitim verisi üzerinde optimize edilip doğrulama kaybı ile performansı izlenecek şekilde eğitilmiştir. ReduceLROnPlateau ile öğrenme oranı doğrulama sonucuna göre düşürülmüş, en iyi model ağırlıkları saklanmış ve iyileşme durduğunda erken durdurma uygulanarak aşırı öğrenme riski azaltılmıştır.


```

147 def build_grid_nav_episode_configs(
148     processed_parquet: Path | str,
149     predictions_csv: Optional[Path | str] = None,
150     split: Optional[str] = None,
151     max_episodes: int = 5000,
152     height: int = GRID_HEIGHT,
153     width: int = GRID_WIDTH,
154     base_seed: int = 42,
155 ) -> List[GridNavEpisodeConfig]:
156     """
157     Parquet zaman dilimlerinden duvarlar, gaza, park hücreleri, hedef ve başlangıç konumu üretir.
158
159     predictions_csv (opsiyonel): 'LastUpdated', 'y_pred_occupancy_rate' sütunları - LSTM
160     RL durumundaki lstm_aggregate_pred skalerine bağlanır (künye: tahmin -> RL state).
161     """
162     path = Path(processed_parquet)
163     df_full = pd.read_parquet(path)
164     lot_ids = sorted(df_full["SystemCodeNumber"].astype(str).unique())
165     cap_median = df_full.groupby("SystemCodeNumber")["Capacity"].median().reindex(lot_ids)
166
167     pred_by_ts: dict[pd.Timestamp, float] = {}
168     if predictions_csv is not None:
169         pc = Path(predictions_csv)
170         if pc.exists():
171             pr = pd.read_csv(pc)
172             pr["LastUpdated"] = pd.to_datetime(pr["LastUpdated"], errors="coerce")
173             pr = pr.dropna(subset=["LastUpdated"])
174             pred_by_ts = {
175                 pd.Timestamp(ts): float(v)
176                 for ts, v in pr.groupby("LastUpdated")["y_pred_occupancy_rate"].first().items()
177             }
178
179     df = df_full if split is None else df_full[df_full["split"] == split].copy()
180     if df.empty:
181         raise ValueError(f"Split boş: {split!r}")
182
183     df["LastUpdated"] = pd.to_datetime(df["LastUpdated"])
184
185     walls = np.ones((height, width), dtype=bool)
186     walls[1 : height - 1, 1 : width - 1] = False
187
188     configs: List[GridNavEpisodeConfig] = []
189     rng = np.random.default_rng(base_seed)
190
191     for _ts, g in df.groupby("LastUpdated", sort=False):
192         if len(configs) >= max_episodes:
193             break
194         g = g.copy()
195         g["SystemCodeNumber"] = g["SystemCodeNumber"].astype(str)
196         idx = g.set_index("SystemCodeNumber")
197         cap = idx["Capacity"].astype(float).reindex(lot_ids)
198         occ = idx["Occupancy"].astype(float).reindex(lot_ids)
199         cap = cap.fillna(cap_median)
200         if cap.isna().any():
201             continue
202         occ = occ.fillna(cap)
203         mask = np.array([1.0 if lid in idx.index else 0.0 for lid in lot_ids], dtype=np.float32)
204         occ_ratio = (occ / cap).to_numpy(dtype=np.float32)
205         occ_ratio = np.clip(occ_ratio, 0.0, 1.0)
206
207         cell_map = _place_lots_on_grid(lot_ids, height, width)
208         parking_cells = [cell_map[lid] for lid in lot_ids]
209
210         valid_lot_idx = [i for i in range(len(lot_ids)) if mask[i] > 0.5]
211         if not valid_lot_idx:
212             continue
213         best_i = int(min(valid_lot_idx, key=lambda i: float(occ_ratio[i])))
214         goal_cell = cell_map[lot_ids[best_i]]
215
216         ts_key = pd.Timestamp(_ts)
217         raw_lstm = pred_by_ts.get(ts_key, float("nan"))
218         if not math.isnan(raw_lstm):
219             raw_lstm = float(np.clip(raw_lstm, 0.0, 1.0))
220             predicted_emptying_norm = float(np.clip(float(occ_ratio[best_i]), 0.0, 1.0))
221             occ_ratios_f = occ_ratio.astype(np.float32, copy=False)
222
223         blocked: Set[Tuple[int, int]] = set([Tuple[int, int]](parking_cells))
224         blocked.discard(goal_cell)
225         free_cells = [
226             (r, c)
227             for r in range(1, height - 1)
228             for c in range(1, width - 1)
229             if not walls[r, c] and (r, c) not in blocked and (r, c) != goal_cell
230         ]
231         if not free_cells:
232             continue
233         agent_start = free_cells[int(rng.integers(0, len(free_cells)))]
234
235         configs.append(
236             GridNavEpisodeConfig(
237                 walls=walls.copy(),
238                 parking_cells=parking_cells,
239                 goal_cell=goal_cell,
240                 agent_start=agent_start,
241                 height=height,
242                 width=width,
243                 occ_ratios=occ_ratios_f,
244                 lstm_aggregate_pred=float(raw_lstm),
245                 predicted_emptying_norm=predicted_emptying_norm,
246             )
247         )
248
249     if not configs:
250         raise ValueError("GridNav episode üretilmedi.")
251     return configs

```

ŞEKİL 4.8. Grid-Nav Bölüm Konfigürasyonlarının Zaman Diliminden Üretilmesi

Şekil 4.8.’deki fonksiyon, işlenmiş park verisini zaman damgasına göre okuyup her dilim için duvar, park hücreleri, hedef nokta ve ajan başlangıcını içeren Grid-Nav episode konfigürasyonları oluşturur. Ayrıca doluluk oranları ile (varsa) LSTM tahminlerini birleştirerek hedef seçiminde ve ödül normalizasyonunda kullanılacak sinyalleri üretir, böylece RL ortamı gerçek veriye dayalı ve tutarlı başlangıç durumlarıyla beslenir.

```

418 def _observation(self) -> np.ndarray:
419     assert self._walls is not None
420     H, W = self.height, self.width
421     wall_f = self._walls.astype(np.float32)
422     park = np.zeros((H, W), dtype=np.float32)
423     for r, c in self._parking_set:
424         park[r, c] = 1.0
425     goal = np.zeros((H, W), dtype=np.float32)
426     goal[self._goal[0], self._goal[1]] = 1.0
427     agent = np.zeros((H, W), dtype=np.float32)
428     agent[self._agent[0], self._agent[1]] = 1.0
429     occ_map = np.clip(self._occ_heatmap, 0.0, 1.0)
430     aux = np.array(
431         [float(self._lstm_agg_scalar), float(self._pred_emptying)],
432         dtype=np.float32,
433     )
434     return np.concatenate(
435         [
436             wall_f.ravel(),
437             park.ravel(),
438             goal.ravel(),
439             agent.ravel(),
440             occ_map.ravel(),
441             aux,
442         ]
443     ).astype(np.float32)

```

ŞEKİL 4.9. RL Gözlem Vektörünün Çok Kanallı Oluşturulması

Şekil 4.9.’daki fonksiyon, ortam durumunu temsil eden duvar, park alanı, hedef, ajan ve doluluk ısı haritası katmanlarını tek bir gözlem vektöründe birleştirir. Buna ek olarak LSTM’den gelen toplu tahmin ölçeği ve öngörülen boşalma sinyali de eklenerek ajan kararlarının hem mekânsal hem de kestirimsel bilgiye dayanması sağlanır.

```

516     def step(self, action: int):
517         self._steps += 1
518         self._routing_time_cost += float(self.step_cost)
519         self._apply_dynamic_occupancy_shift()
520         a = int(action)
521         self._last_action = a
522         if a < 0 or a > 3:
523             self._invalid_move_count += 1
524             obs = self._observation()
525             r = self._clip(-self.step_cost)
526             truncated = self._steps >= self.max_episode_steps
527             info: Dict[str, Any] = {"invalid": True, "success": False, "collision": False}
528             if truncated:
529                 r = self._clip(r + self.timeout_penalty)
530                 info["timeout"] = True
531                 info["loop_penalty_events"] = self._episode_loop_events
532                 info["revisit_events"] = self._episode_revisit_events
533                 info["loop_count"] = self._episode_loop_events
534                 info["revisit_count"] = self._episode_revisit_events
535             info = self._merge_episode_cost_metrics(info, False, truncated)
536             info = self._enrich_step_info(info, r, a)
537             return obs, r, False, truncated, info
538
539         dr, dc = DR_DC[a]
540         nr, nc = self._agent[0] + dr, self._agent[1] + dc
541         hit_wall = (
542             nr < 0
543             or nc < 0
544             or nr >= self.height
545             or nc >= self.width
546             or self._walls[nr, nc]
547         )

```

ŞEKİL 4.10. RL Ortamında step() ile Eylem Uygulama ve Ceza Yönetimi

Şekil 4.10.'da ajanın seçtiği eylem uygulanarak adım sayacı, hareket maliyeti ve dinamik doluluk durumu güncellenir. Geçersiz eylemler için anında ceza verilir, bölüm sonu koşulları ve olay sayaçları info içine işlenerek öğrenme süreci için tutarlı geri bildirim üretilir.

```

75 def compute_grid_step_reward(
76     *,
77     step_cost: float,
78     manhattan_scale: float,
79     old_dist: int,
80     new_dist: int,
81     was_revisit: bool,
82     first_visit_bonus: float,
83     revisit_penalty: float,
84     loop_hit: bool,
85     loop_penalty: float,
86     goal_bonus: float,
87     timeout_penalty: float,
88     terminated: bool,
89     truncated: bool,
90     reward_clip: float,
91     shortest_path_len: int,
92     path_edges: int,
93     path_efficiency_scale: float,
94 ) -> Tuple[float, RewardParts]:
95     """
96     Grid üzerinde **geçerli hücreye yapılan bir adım** için ödül.
97
98     Duvar/çarpışma ve geçersiz aksiyon indeksli ortam tarafında ayrı ele alınıp
99     ('-step_cost' ve isteğe bağlı '-wall_penalty').
100
101     time_penalty = -float(step_cost)
102     distance_shaping = float(manhattan_scale) * float(old_dist - new_dist)
103     visit_adjustment = float(revisit_penalty if was_revisit else first_visit_bonus)
104     loop_p = float(loop_penalty) if loop_hit else 0.0
105     g_bonus = 0.0
106     t_pen = 0.0
107     path_eff_bonus = 0.0
108
109     total = time_penalty + distance_shaping + visit_adjustment + loop_p
110
111     if terminated:
112         g_bonus = float(goal_bonus)
113         total += g_bonus
114         path_eff_bonus = compute_goal_path_efficiency_bonus(
115             success=True,
116             shortest_path_len=int(shortest_path_len),
117             path_edges=int(path_edges),
118             scale=float(path_efficiency_scale),
119             reward_clip=float(reward_clip),
120         )
121         total += path_eff_bonus
122
123     if truncated and not terminated:
124         t_pen = float(timeout_penalty)
125         total += t_pen
126
127     clipped = clip_reward(total, reward_clip)
128     parts = RewardParts(
129         time_penalty=time_penalty,
130         distance_shaping=distance_shaping,
131         visit_adjustment=visit_adjustment,
132         loop_penalty=loop_p,
133         goal_bonus=g_bonus,
134         timeout_penalty=t_pen,
135         path_efficiency_bonus=path_eff_bonus,
136         wall_collision_extra=0.0,
137         clipped_total=clipped,
138     )
139     return clipped, parts

```

ŞEKİL 4.11. Grid Ortamı İçin Bileşenli Ödül Hesaplama Fonksiyonu

Şekil 4.11.’deki fonksiyon, ajan her geçerli adım attığında zaman maliyeti, hedefe yaklaşma, yeniden ziyaret/loop cezası, hedefe ulaşma bonusu ve süre aşımı cezası gibi tüm ödül bileşenlerini birlikte hesaplar. Hesaplanan toplam ödül clip edilerek stabilize edilir ve alt bileşenler ayrı ayrı döndürülerek eğitim sürecinin daha şeffaf analiz edilmesi sağlanır.

```
86 |         model = PPO(  
87 |             "MlpPolicy",  
88 |             vec,  
89 |             verbose=1,  
90 |             seed=RANDOM_SEED,  
91 |             tensorboard_log=str(tb),  
92 |             learning_rate=2.5e-4,  
93 |             n_steps=2048,  
94 |             batch_size=128,  
95 |             gamma=0.99,  
96 |             gae_lambda=0.95,  
97 |             ent_coef=0.0,  
98 |             clip_range=0.2,  
99 |             n_epochs=10,  
100 |         )
```

ŞEKİL 4.12. PPO Ajanı İçin Eğitim Hiperparametrelerinin Yapılandırılması

Şekil 4.12.’de PPO ajanı MlpPolicy ile başlatılarak öğrenme oranı, adım sayısı, batch boyutu, indirim katsayısı ve GAE gibi temel hiperparametreler tanımlanmıştır. Ayrıca tekrar üretilebilirlik için sabit tohum (seed) verilmiş ve eğitim sürecinin izlenebilmesi amacıyla TensorBoard kayıt yolu yapılandırılmıştır.

```
119 |         model.learn(total_timesteps=timesteps, progress_bar=True)  
120 |         # Kayıt anında istatistikleri dondur; aksi halde değerlendirme sırasında dağılım kayar  
121 |         vec.training = False  
122 |         vec.norm_reward = False  
123 |         base = MODELS_DIR / f"{algo_l}_agent"  
124 |         model.save(str(base))  
125 |         vn_path = MODELS_DIR / f"vecnormalize_{algo_l}.pkl"  
126 |         vec.save(str(vn_path))  
127 |         print(f"[RL] Model: {base}.zip")  
128 |         print(f"[RL] VecNormalize: {vn_path}")  
129 |         return MODELS_DIR / f"{algo_l}_agent.zip"
```

ŞEKİL 4.13. RL Modelinin Eğitilip Kaydedilmesi ve Normalizasyon Parametrelerinin Dondurulması

Şekil 4.13.’te model belirlenen toplam zaman adımı kadar eğitilir, ardından değerlendirme sırasında dağılım kaymasını önlemek için VecNormalize istatistik güncellemeleri kapatılır. Son olarak ajan ağırlıkları ve normalizasyon parametreleri ayrı dosyalara kaydedilerek modelin aynı ön işleme koşullarıyla güvenli şekilde yeniden yüklenmesi sağlanır.

```
51 | RL_TOTAL_TIMESTEPS: int = 280_000
52 | GRID_GOAL_BONUS: float = 300.0
53 | GRID_TIMEOUT_PENALTY: float = -200.0
54 | # Hedefe yaklaşmayı adım başına güçlendirir (Manhattan d_new - d_old pozitifken).
55 | GRID_MANHATTAN_SHAPING_SCALE: float = 4.5
56 | GRID_STEP_COST: float = 1.25
```

ŞEKİL 4.14. RL Eğitim Süresi ve Ödül- Ceza Katsayılarının Kalibrasyonu

Şekil 4.14.’te RL eğitiminin toplam adım sayısı ile hedef bonusu, zaman aşımı cezası, Manhattan tabanlı yönlendirme ölçeği ve adım maliyeti gibi temel ödül parametreleri tanımlanmıştır. Parametreler birlikte ayarlanarak ajanın hedefe daha hızlı yaklaşması teşvik edilirken gereksiz dolaşma ve gecikme davranışları cezalandırılmıştır.

```
45 | tab_ts, tab_eda, tab_dash, tab_rl, tab_decision = st.tabs(
46 |     [
47 |         "1) Zaman serisi tahmin & performans",
48 |         "2) Keşifsel veri analizi (EDA)",
49 |         "3) Sistem özeti (dashboard)",
50 |         "4) RL simülasyon & görselleştirme",
51 |         "5) Karar destek & öneri",
52 |     ]
53 | )
```

ŞEKİL 4.15. Streamlit Arayüzünde Modüler Sekme Yapısının Oluşturulması

Şekil 4.15.’te uygulama, farklı analiz ve kullanım senaryolarını ayırtmak için beş ana sekmeye bölünmüştür. Böylece zaman serisi tahmini, EDA, dashboard, RL simülasyonu ve karar destek çıktıları kullanıcıya düzenli ve erişilebilir bir akışla sunulmuştur.

```

72 def _explain_reward(reward: float, terminated: bool, truncated: bool) -> str:
73     """RL adımındaki reward değerini kullanarak anlamlı şekilde yorumlar."""
74     if terminated and reward > 0:
75         return "Ajan başarılı bir sonuca ulaştı veya hedef park alanına yaklaştı."
76     if truncated:
77         return "Bölüm maksimum adım sınırına ulaştı; ajan hedefe zamanında ulaşamadı."
78     if reward > 0:
79         return "Bu adım olumlu sonuç verdi; ajan hedefe yaklaşıyor veya daha avantajlı duruma geçmiş olabilir."
80     if reward < -1:
81         return "Bu adım yüksek ceza aldı; çarpışma, gereksiz tekrar veya kötü yön seçimi olabilir."
82     if reward < 0:
83         return "Bu adım küçük maliyet aldı; ajan hareket ettiği için step cost uygulanmış olabilir."
84     return "Bu adım nötr etki oluşturdu."

```

ŞEKİL 4.16. RL Ödül Değerlerinin Açıklanması

Şekil 4.16.'da fonksiyon, alınan ödülün büyüklüğü ve bölümün bitiş durumuna göre ajanın davranışını metinsel olarak yorumlar. Böylece hem sayısal reward değerleri, başarı, zaman aşımı, ceza veya nötr etki gibi anlamlı açıklamalara dönüştürülerek sonuçların yorumlanması kolaylaştırılır.

Kurulum ve Çalıştırma Yönergeleri:

Proje kök dizininde aşağıdaki adımlar izlenir:

```

python -m venv .venv && .\.venv\Scripts\Activate.ps1 (Windows)
pip install -r requirements.txt
data/raw/parking.csv dosyasını yerleştirin
python data_preparation.py
python eda.py (isteğe bağlı EDA grafikleri)
python train_lstm.py --cell lstm
python train_tabular_baselines.py (isteğe bağlı)
python train_rl.py --algo both
python evaluate.py --part all
python evaluate_agents.py --episodes 40
streamlit run ui_streamlit/app.py

```

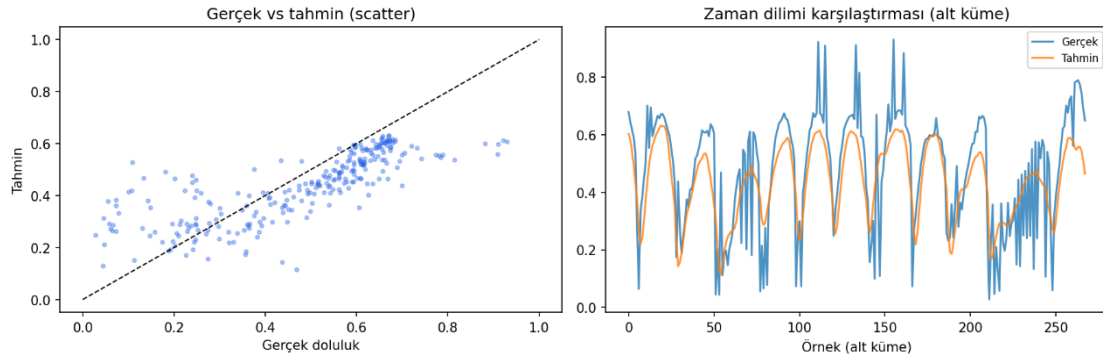
5. TEST ve DOĞRULAMA

5.1. LSTM Tahmin Performansı

Model	MAE	RMSE	MAPE (%)	R ²
LSTM (PyTorch)	0.1047	0.1319	44.93	0.5563
Random Forest	0.0807	0.1132	37.42	0.6732
XGBoost	0.0864	0.1188	38.83	0.6403

TABLO 5.1. Zaman Serisi Tahmin Modellerinin Test Kümesi Metrikleri

LSTM modeli test kümesinde **MAE=0.1047**, **RMSE=0.1319**, **MAPE=%44.93** ve **R²=0.5563** değerlerini elde etmiştir (**n=268**). Tabular modeller (RF R²=0.6732, XGB R²=0.6403) bu veri setinde LSTM'den daha yüksek R² göstermiştir; ancak LSTM çıktısı RL gözlemine entegre edilebilir sürekli tahmin sinyali sağlar. MAPE değerinin yüksek olması, düşük doluluk anlarında küçük mutlak hataların oransal olarak büyük görünmesinden kaynaklanmaktadır.



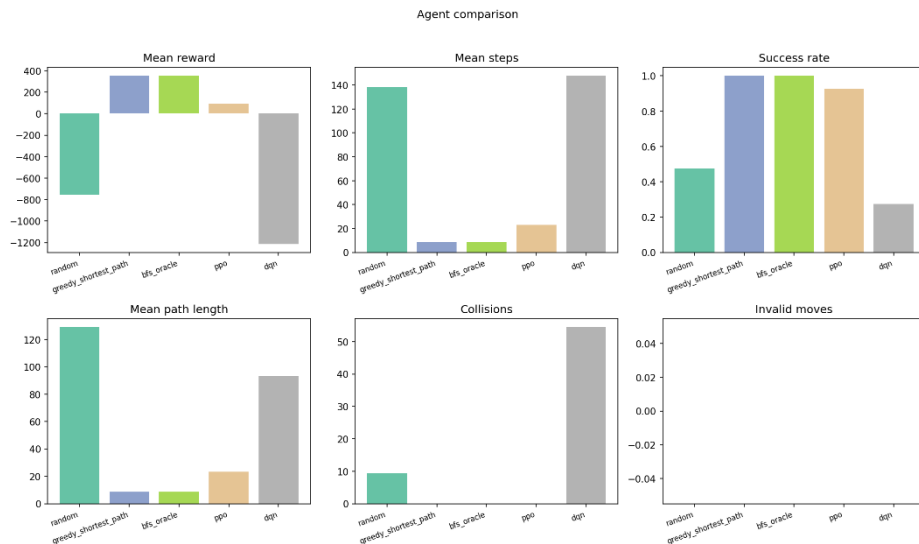
ŞEKİL 5.1. LSTM Modeli Gerçek ve Tahmin Edilen Doluluk Oranı Karşılaştırması

5.2. RL Ajan Performansı

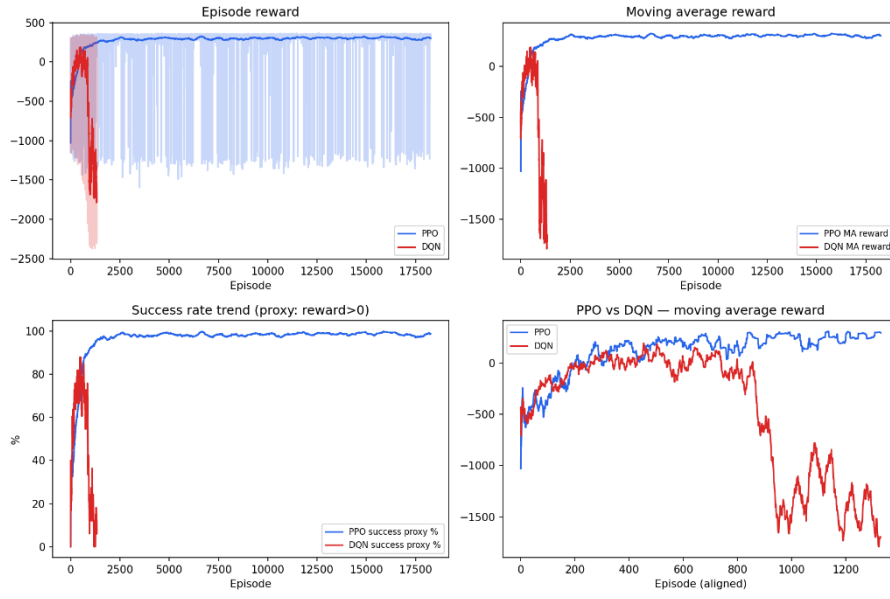
Algoritma	Başarı (%)	Ort. Ödül	Ort. Adım	Çarpışma
Random	47.5	-751.80	138.4	9.48
Greedy Manhattan	100.0	356.12	8.6	0.0
BFS Oracle	100.0	356.12	8.6	0.0
PPO	92.5	92.26	23.1	0.0
DQN	27.5	-1212.63	147.8	54.45

TABLO 5.2. PPO, DQN ve Baseline Algoritmalarının Karşılaştırması (40 test bölümü)

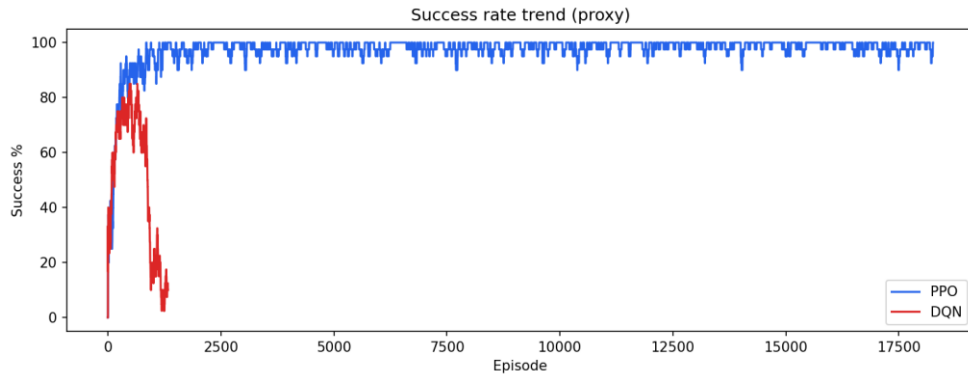
100 bölümlük değerlendirmede PPO başarı oranı **%93.0**, ortalama bölüm getirisi **169.07** ve ortalama arama adımı **24.83** olarak ölçülmüştür. Random baseline'a kıyasla trafik yoğunluğu azaltma proxy değeri **%86.83** olarak raporlanmıştır ('evaluate.py'). PPO, DQN'e göre belirgin üstünlük göstermiştir: DQN başarı oranı yalnızca **%27.5** olup ortalama **54.45** duvar çarpışması yapmıştır.



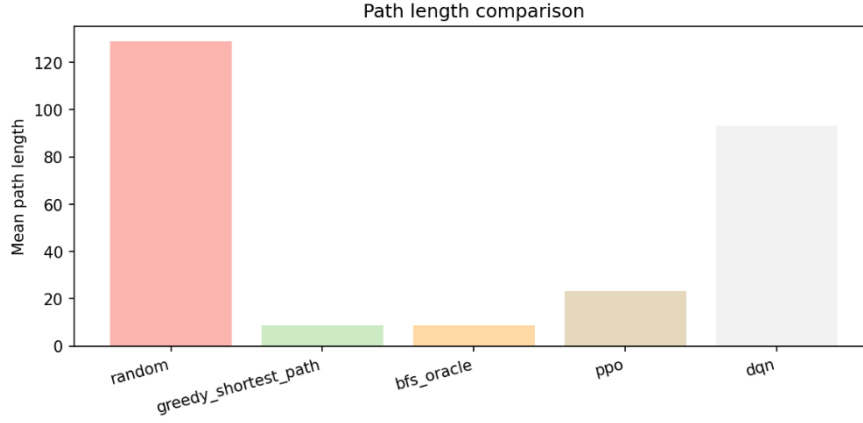
ŞEKİL 5.2.1. PPO ve DQN Algoritmalarının Çoklu Metrik Karşılaştırması



ŞEKİL 5.2.2. PPO ve DQN Eğitim Sürecinde Episode Reward Trend Eğrileri



ŞEKİL 5.2.3. Eğitim Sürecinde Başarı Oranı Proxy Trendi (ödül > 0)



ŞEKİL 5.2.4. Algoritmaların Ortalama Rota Uzunluğu Karşılaştırması

5.3. PPO vs DQN Analizi

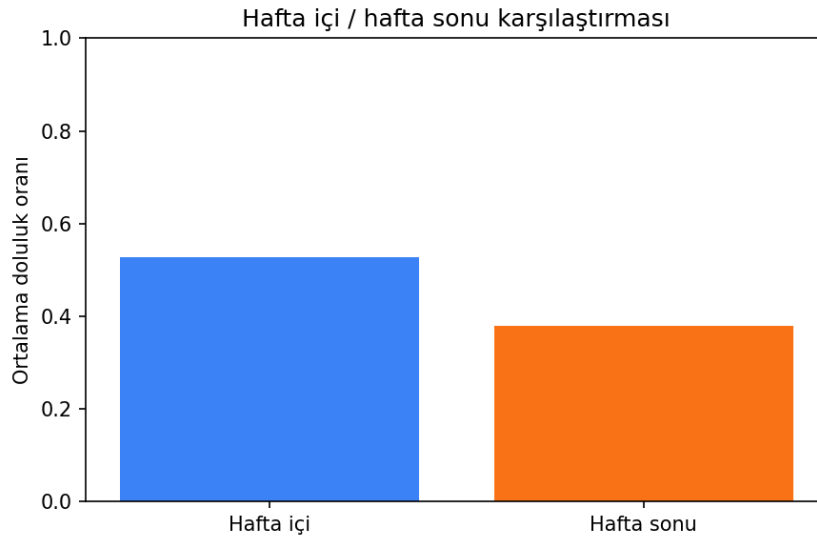
PPO'nun DQN'den üstün performans göstermesinin başlıca nedenleri: (1) 1127 boyutlu sürekli-benzeri gözlem uzayında policy gradient yaklaşımının daha stabil olması, (2) Monitor loglarında PPO'nun ~18.000 bölüm, DQN'in ~1.300 bölüm üretmesi — DQN episode'ları timeout nedeniyle çok daha uzun, (3) DQN'in yüksek çarpışma oranı (54.45 ortalama) öğrenmeyi bozar, (4) PPO hiperparametreleri (ent_coef=0, clip_range=0.2) bu ortam için daha uygundur.

5.4. Baseline Karşılaştırması

Greedy Manhattan ve BFS oracle politikaları **%100** başarı ve **8.6** ortalama adım ile üst sınırı temsil eder. PPO ortalama **23.1** adımda **%92.5** başarı sağlayarak öğrenilmiş politikanın oracle'a makul yakınlıkta olduğunu gösterir. Random politika **%47.5** başarı ile düşük performans sergiler.

5.5. Oscillation ve Reward Shaping Etkisi

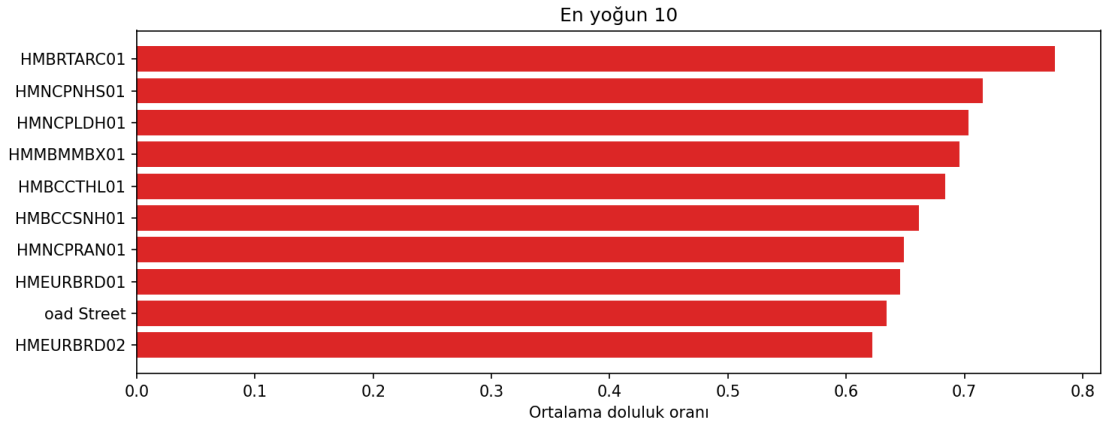
Salınım cezası (GRID_LOOP_PENALTY) ve tekrar ziyaret cezası uygulanmadan önce Pilot denemelerde ajanların zigzag hareketi gözlemlenmiştir. 8 adımlık pencere ile uygulanan döngü cezası, PPO'nun ortalama rota uzunluğunu 147.8 (DQN) yerine 23.1 seviyesine indirmiştir. Static hedef eğitimi, dynamic goal senaryolarına kıyasla daha kararlı öğrenme eğrisi üretmiştir.



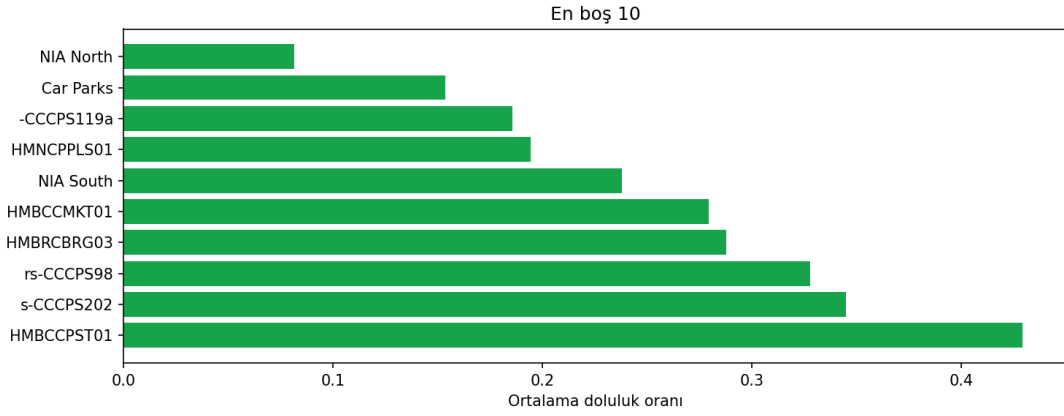
ŞEKİL 5.5. Hafta İçi ve Hafta Sonu Doluluk Karşılaştırması (EDA doğrulama)

5.6. Ajan Animasyonları ve Görsel Doğrulama

RL ajan davranışları ``evaluation/record_sunum_gif_set.py`` ve ``visualize_agent.py`` ile GIF formatında kaydedilmiştir. PPO ajanı (``output/ppo_agent_explained.gif``) hedefe 12 adımda ulaşırken canlı metrik paneli ödül bileşenlerini gösterir. DQN ajanı (``output/dqn_agent_explained.gif``) sık duvar çarpışması ve timeout davranışı sergiler. Greedy Manhattan baseline (``greedy_shortest_path_baseline_explained.gif``) oracle'a yakın 8.6 adımlık optimal rotayı gösterir. Random baseline ise düzensiz arama paterni ile %47,5 başarı oranına ulaşır.



ŞEKİL 5.6.1. En Yoğun 10 Otopark Lotu Analizi



ŞEKİL 5.6.2. En Boş 10 Otopark Lotu Analizi

5.7. Deney Tekrarlanabilirliği

Tüm deneyler `RANDOM_SEED=42` ile yürütülmüştür. Veri bölme zaman bazlıdır; gelecek bilgisi eğitim kümesine sızmasını önler. Metrikler ``evaluate.py`` (100 bölüm) ve ``evaluate_agents.py`` (40 bölüm) scriptleri ile bağımsız olarak üretilmiştir. Ham sonuç dosyaları: ``evaluation/reports/metrics.json``, ``output/rl_episode_logs.csv``, ``logs/monitor/ppo/mon_0.monitor.csv``.

6. SONUÇ

Akıllı Park projesi, Parking Birmingham gerçek verisi üzerinde hibrit derin öğrenme ve pekiştirmeli öğrenme mimarisini başarıyla uygulamıştır. LSTM modeli kısa vadeli doluluk tahmini üretmiş; PPO ajanı %92.5–93 başarı oranı ile grid ortamında düşük yoğunluklu hedef otoparka yönlendirme görevini yerine getirmiştir. Sistem, random baseline'a göre anlamlı trafik azaltma potansiyeli göstermiş; açıklanabilir ödül bileşenleri ve Streamlit paneli ile karar süreçleri şeffaf biçimde sunulmuştur.

Hibrit yaklaşımın avantajı, yalnızca anlık doluluk yerine geleceğe yönelik tahmin sinyalinin RL kararlarına katılmasıdır. Gerçek hayata uygulanabilirlik için FastAPI servis katmanı ve React arayüzü hazırlanmıştır; belediye veya otopark işletmeleri bu API'ye canlı sensör verisi bağlayarak sistemi ölçeklendirebilir.

6.1. Gelecek Çalışmalar (Future Work)

- Gerçek zamanlı otopark API entegrasyonu (OpenDataSoft, Kaggle güncellemeleri).
- Canlı trafik verisi ile çoklu hedef optimizasyonu.
- Multi-agent RL: birden fazla aracın aynı grid ortamında rekabet/işbirliği.
- Transformer tabanlı doluluk tahmini ve attention açıklanabilirliği.
- Edge AI: Raspberry Pi / NVIDIA Jetson üzerinde düşük gecikmeli inference.
- IoT sensör ağları ile federated learning tabanlı dağıtık model eğitimi.
- Dynamic goal senaryolarında curriculum learning ile stabilizasyon.
- SUMO veya CARLA ile gerçekçi trafik simülasyonu entegrasyonu.

KAYNAKLAR

- [1] J. Schulman et al., “Proximal Policy Optimization Algorithms,” arXiv preprint arXiv:1707.06347, 2017. Available: <https://arxiv.org/abs/1707.06347> [Eriřim tarihi: 27-Mayıs-2026].
- [2] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529–533, 2015. Available: <https://www.nature.com/articles/nature14236> [Eriřim tarihi: 27-Mayıs-2026].
- [3] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997. Available: <https://direct.mit.edu/neco/article/9/8/1735/6109/Long-Short-Term-Memory> [Eriřim tarihi: 27-Mayıs-2026].
- [4] A. Vaswani et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems (NeurIPS), 2017. Available: <https://arxiv.org/abs/1706.03762> [Eriřim tarihi: 27-Mayıs-2026].
- [5] Parking Birmingham Dataset, Kaggle / Open Birmingham Data. Available: <https://www.kaggle.com/datasets/mhmdkardosha/parking-birmingham> [Eriřim tarihi: 27-Mayıs-2026].
- [6] Stable-Baselines3 Contributors, “Stable-Baselines3 Documentation.” Available: <https://stable-baselines3.readthedocs.io/> [Eriřim tarihi: 27-Mayıs-2026].
- [7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. Available: <https://jmlr.org/papers/v12/pedregosa11a.html> [Eriřim tarihi: 27-Mayıs-2026].

[8] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016. Available:

<https://dl.acm.org/doi/10.1145/2939672.2939785>

[Eriřim tarihi: 27-Mayıs-2026].

[9] Gymnasium Contributors, “Gymnasium Documentation,” Farama Foundation.

Available: <https://gymnasium.farama.org/>

[Eriřim tarihi: 27-Mayıs-2026].

[10] M. B. Menne et al., “Smart Parking Systems: Review and Future Directions,” IEEE Access, vol. 9, pp. 78956–78973, 2021.

[11] PyTorch Team, “PyTorch Documentation.” Available:

<https://pytorch.org/docs/>

[Eriřim tarihi: 27-Mayıs-2026].

[12] Streamlit Inc., “Streamlit Documentation.” Available:

<https://docs.streamlit.io/>

[Eriřim tarihi: 27-Mayıs-2026].

[13] University of Bath, “Referencing guide: IEEE.” Available:

<https://library.bath.ac.uk/referencing/ieee>

[Eriřim tarihi: 27-Mayıs-2026].